

Episodio III

Revenge of the Cth



I've brought peace, freedom, justice and security to my new Operating System

Cátedra de Sistemas Operativos

Trabajo práctico Cuatrimestral

-1C2025 -
Versión 1.0

Índice

Índice	2
Historial de Cambios	4
Objetivos del Trabajo Práctico	5
Características	5
Evaluación del Trabajo Práctico	5
Deployment y Testing del Trabajo Práctico	6
Aclaraciones	6
Definición del Trabajo Práctico	7
¿Qué es el trabajo práctico y cómo empezamos?	7
Arquitectura del sistema	8
Aclaración importante	8
Módulo: Kernel	9
Lineamiento e Implementación	9
Diagrama de estados	9
PCB	10
Planificador de Largo Plazo	10
Creación de procesos	10
FIFO	10
Proceso más chico primero	10
Finalización de procesos	11
Planificador de Corto Plazo	11
FIFO	11
SJF sin Desalojo	11
SJF con Desalojo	11
Ejecución	11
Syscalls	12
Procesos	12
Memoria	12
Entrada Salida	12
Planificador de Mediano Plazo	13
Logs mínimos y obligatorios	13
Archivo de Configuración	13
Ejemplo de Archivo de Configuración	14
Módulo: IO	15
Lineamiento e Implementación	15
Logs mínimos y obligatorios	15
Archivo de Configuración	15
Ejemplo de Archivo de Configuración	15
Módulo: CPU	16
Lineamiento e Implementación	16

Ciclo de Instrucción	16
Fetch	16
Decode	16
Ejemplos de instrucciones a interpretar	16
Execute	17
Check Interrupt	17
MMU	17
TLB	18
Caché de páginas	18
Logs mínimos y obligatorios	19
Archivo de Configuración	19
Ejemplo de Archivo de Configuración	20
Módulo: Memoria + SWAP	21
Lineamiento e Implementación	21
Memoria de Sistema	21
Archivos de pseudocódigo	21
Memoria de Usuario	21
Esquema de memoria	21
Consideraciones teóricas del esquema	22
Estructuras	22
Métricas por proceso	22
Comunicación con Kernel y CPU	23
Inicialización del proceso	23
Suspensión de proceso	23
Finalización de proceso	23
Acceso a tabla de páginas	23
Acceso a espacio de usuario	23
Leer Página completa	23
Actualizar página completa	24
Memory Dump	24
Manejo de SWAP	24
Logs mínimos y obligatorios	24
Archivo de Configuración	25
Ejemplo de Archivo de Configuración	25
Descripción de las entregas	26
Check de Control Obligatorio 1: Conexión inicial	26
Check de Control Obligatorio 2: Ejecución Básica	26
Check de Control Obligatorio 3: CPU Completa y Memoria	27
Entregas Finales	27

Historial de Cambios

v1.0 (08/04/2025) *Publicación del enunciado*

Objetivos del Trabajo Práctico

Mediante la realización de este trabajo se espera que el alumno:

- Adquiera conceptos prácticos del uso de las distintas herramientas de programación e interfaces (APIs) que brindan los sistemas operativos.
- Entienda aspectos del diseño de un sistema operativo.
- Afirme diversos conceptos teóricos de la materia mediante la implementación práctica de algunos de ellos.
- Se familiarice con técnicas de programación de sistemas, como el empleo de makefiles, archivos de configuración y archivos de log.
- Conozca con grado de detalle la operatoria de Linux mediante la utilización de un lenguaje de programación de relativamente bajo nivel como C.

Características

- Modalidad: grupal (5 integrantes $\pm$ 0) y obligatorio
- Fecha de comienzo: 08/04/2025
- Fecha de primera entrega: 12/07/2025
- Fecha de segunda entrega: 19/07/2025
- Fecha de tercera entrega: 02/08/2025
- Lugar de corrección: Laboratorio de Sistemas - Medrano.

Evaluación del Trabajo Práctico

El trabajo práctico consta de una evaluación en 2 etapas.

La primera etapa consistirá en las pruebas de los programas desarrollados en el laboratorio. Las pruebas del trabajo práctico se subirán oportunamente y con suficiente tiempo para que los alumnos puedan evaluarlas con antelación. Queda aclarado que para que un trabajo práctico sea considerado evaluable, el mismo debe proporcionar registros de su funcionamiento de la forma más clara posible.

La segunda etapa se dará en caso de aprobada la primera y constará de un coloquio, con el objetivo de afianzar los conocimientos adquiridos durante el desarrollo del trabajo práctico y terminar de definir la nota de cada uno de los integrantes del grupo, por lo que se recomienda que la carga de trabajo se distribuya de la manera más equitativa posible.

Cabe aclarar que el trabajo equitativo no asegura la aprobación de la totalidad de los integrantes, sino que cada uno tendrá que defender y explicar tanto teórica como prácticamente lo desarrollado y aprendido a lo largo de la cursada.

La defensa del trabajo práctico (o coloquio) consta de la relación de lo visto durante la teoría con lo implementado. De esta manera, una implementación que contradiga lo visto en clase o lo escrito en el documento *es motivo de desaprobación del trabajo práctico*. Esta etapa al ser la conclusión del todo el trabajo realizado durante el cuatrimestre no es recuperable.

Deployment y Testing del Trabajo Práctico

Al tratarse de una plataforma distribuida, los procesos involucrados podrán ser ejecutados en diversas computadoras. La cantidad de computadoras involucradas y la distribución de los diversos procesos en estas será definida en cada uno de los tests de la evaluación y es posible cambiar la misma en el momento de la evaluación. Es responsabilidad del grupo automatizar el despliegue de los diversos procesos con sus correspondientes archivos de configuración para cada uno de los diversos tests a evaluar.

Todo esto estará detallado en el documento de pruebas que se publicará cercano a la fecha de Entrega Final. Archivos y programas de ejemplo se pueden encontrar en el repositorio de la cátedra.

Finalmente, es mandatoria la lectura y entendimiento de las [Normas del Trabajo Práctico](#) donde se especifican todos los lineamientos de cómo se desarrollará la materia durante el cuatrimestre.

Aclaraciones

Debido al fin académico del trabajo práctico, los conceptos reflejados son, en general, versiones simplificadas o alteradas de los componentes reales de hardware y de sistemas operativos vistos en las clases, a fin de resaltar aspectos de diseño o simplificar su implementación.

Invitamos a los alumnos a leer las notas y comentarios al respecto que haya en el enunciado, reflexionar y discutir con sus compañeros, ayudantes y docentes al respecto.

Definición del Trabajo Práctico

Esta sección se compone de una introducción y definición de carácter global sobre el trabajo práctico. Posteriormente se explicarán por separado cada uno de los distintos módulos que lo componen, pudiéndose encontrar los siguientes títulos:

- **Lineamiento e Implementación:** Todos los títulos que contengan este nombre representarán la definición de lo que deberá realizar el módulo y cómo deberá ser implementado. La no inclusión de alguno de los puntos especificados en este título puede conllevar a la desaprobación del trabajo práctico.
- **Archivos de Configuración:** En este punto se da un archivo modelo y que es lo mínimo que se pretende que se pueda parametrizar en el proceso de forma simple. En caso de que el grupo requiera de algún parámetro extra, podrá agregarlo.

Cabe destacar que en ciertos puntos de este enunciado se explicarán exactamente cómo deben ser las funcionalidades a desarrollar, mientras que en otros no se definirá específicamente, quedando su implementación a decisión y definición del equipo. Se recomienda en estos casos siempre consultar en el [foro de github](#).

¿Qué es el trabajo práctico y cómo empezamos?

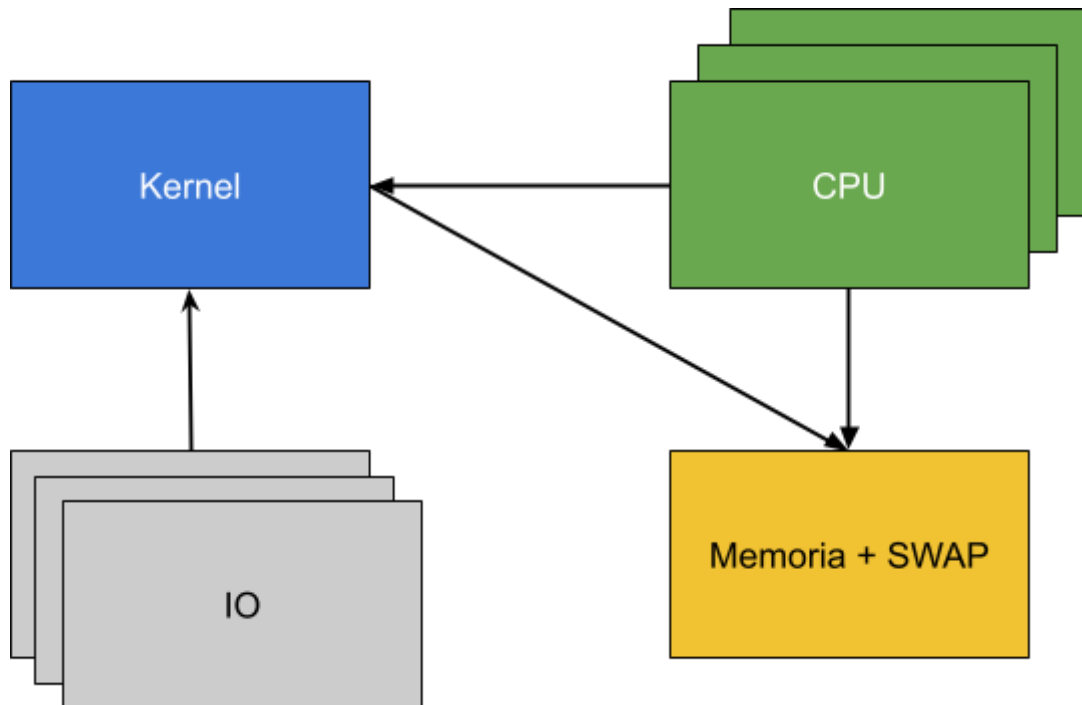
El objetivo del trabajo práctico consiste en desarrollar una solución que permita la simulación de un sistema distribuido, donde los grupos tendrán que planificar procesos, resolver peticiones al sistema y administrar de manera adecuada una memoria bajo los esquemas explicados en sus correspondientes módulos.

Para el desarrollo del mismo se decidió la creación de un sistema bajo la metodología Iterativa Incremental donde se solicitarán en una primera instancia la implementación de ciertos módulos para luego poder realizar una integración total con los restantes.

Recomendamos seguir el lineamiento de los distintos puntos de control que se detallan al final de este documento para su desarrollo. Estos puntos están planificados y estructurados para que sean desarrollados a medida y en paralelo a los contenidos que se ven en la parte teórica de la materia. *Cabe aclarar que esto es un lineamiento propuesto por la cátedra y no implica impedimento alguno para el alumno de realizar el desarrollo en otro orden diferente al especificado.*

Arquitectura del sistema

Este trabajo práctico correrá una serie de módulos los cuales deberán ser capaces de correr en diferentes computadoras y/o máquinas virtuales.



Aclaración importante

Desarrollar únicamente temas de conectividad, serialización, sincronización o el módulo IO es insuficiente para poder entender y aprender los distintos conceptos de la materia. Dicho caso será un motivo de desaprobación directa.

Cada módulo contará con un listado de **logs mínimos y obligatorios** los cuales deberán realizarse utilizando la biblioteca de `so-commons-library` provista por la cátedra y los mismos deberán estar como `LOG_LEVEL_INFO`, pudiendo ser extendidos por necesidad del grupo utilizando `LOG_LEVEL_DEBUG`.

En caso de no cumplir con los logs mínimos y/o tener los logs impresos por pantalla, *se considerará que el TP no es apto para ser evaluado* y por consecuencia el mismo estará *desaprobado*.

Módulo: Kernel

El módulo **Kernel**, en el contexto de nuestro trabajo práctico, será el módulo encargado de la gestión de los procesos que se creen dentro de este sistema simulado, planificando su ejecución en múltiples CPUs mediante diferentes algoritmos, llevando a cabo sus peticiones de entrada/salida hacia múltiples dispositivos (representados por el módulo I/O) e interactuando con el módulo Memoria para su utilización.

Lineamiento e Implementación

Para lograr su propósito, este módulo mantendrá conexiones constantes con una o más instancias del módulo CPU y conexiones efímeras para interactuar con el módulo Memoria.

Para la conexión con cada instancia del módulo CPU, se tendrán 2 sockets: en el primero enviará los procesos a ejecutar y se quedará esperando la respuesta de la CPU a dicha solicitud; y el segundo socket será utilizado para enviarle las notificaciones de interrupciones al momento de trabajar con algoritmos que requieran interrumpir a la CPU.

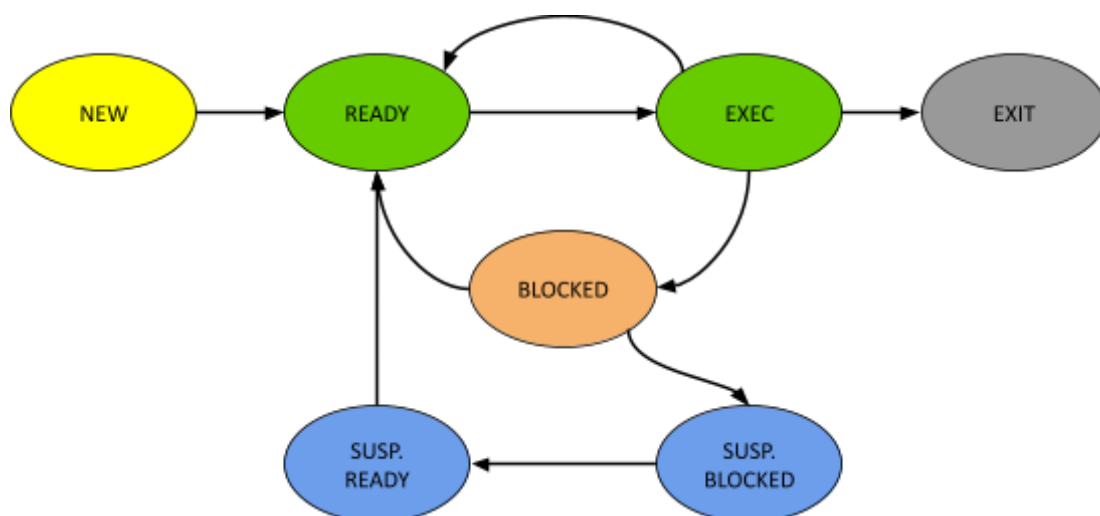
Para las conexiones con el módulo Memoria, el Kernel **deberá crear una nueva conexión para cada petición** que requiera hacerle a la Memoria.

Al iniciar el módulo, se creará un proceso inicial para que este lo planifique y para poder inicializarlo, se requerirá que este módulo reciba dos argumentos adicionales en la función main: el nombre del archivo de pseudocódigo que deberá ejecutar y el tamaño del proceso para ser inicializado en Memoria. Ejemplo de comando de ejecución:

```
➔ ~ ./bin/kernel [archivo_pseudocodigo] [tamaño_proceso] [...args]
```

Diagrama de estados

El kernel utilizará un diagrama de 7 estados para la planificación de los procesos e hilos.



PCB

El PCB será la estructura base que utilizaremos dentro del Kernel para administrar cada uno de los procesos. El mismo deberá contener como mínimo los datos definidos a continuación, que representan la información administrativa.

- **PID:** Identificador del proceso (deberá ser un número entero, único en todo el sistema que arranque en 0).
- **PC:** Program Counter, es un número entero que arranca en 0.
- **ME:** Métricas de Estado, un listado de estados contabilizando la cantidad de veces que el proceso estuvo en cada uno de ellos.
- **MT:** Métricas de tiempo por estado, un listado con los tiempos en los que permaneció el proceso en cada estado.

Planificador de Largo Plazo

El Kernel será el encargado de gestionar las peticiones a la memoria para la creación y eliminación de procesos e hilos.

Creación de procesos

Se tendrá una cola NEW que será administrada mediante un algoritmo definido por archivo de configuración. Estos algoritmos son: FIFO y Proceso mas chico primero (siendo mas chico el que menos memoria solicite).

Al llegar un nuevo proceso a esta cola y la misma esté vacía, se enviará un pedido a Memoria para inicializar el mismo. Si la respuesta es positiva, se pasará el proceso al estado READY y se sigue la misma lógica con el proceso que sigue. Si la respuesta es negativa (ya que la Memoria no tiene espacio suficiente para inicializarlo) se deberá esperar la finalización de otro proceso para volver a intentar inicializarlo.

En cambio, si un proceso llega a esta cola y ya hay otros esperando se debe tener en cuenta el algoritmo definido y verificar si corresponde o no su ingreso.

FIFO

Bajo el algoritmo FIFO, si hay un proceso en NEW que no puede pasar a ready por falta de espacio en la memoria, todos los procesos que lleguen a continuación, deberán esperar a que el primero ingrese.

Proceso más chico primero

En este algoritmo se va a priorizar a los procesos más chicos, por lo tanto, al llegar un nuevo proceso a NEW, independientemente de si hay o no procesos esperando, vamos a consultarle a la memoria si dispone del espacio requerido para iniciar el nuevo proceso y en caso afirmativo, se transiciona el proceso a READY.

Por último, al iniciar el proceso Kernel el algoritmo de Largo plazo debe estar parado (estado STOP) y se deberá esperar un ingreso de un *Enter* por teclado para poder iniciar con la planificación.

Finalización de procesos

Al momento de finalizar un proceso, el Kernel deberá informar a la Memoria la finalización del mismo y luego de recibir la confirmación por parte de la Memoria deberá liberar su PCB asociado e intentar inicializar uno de los que estén esperando, para ello primero se deberá verificar los procesos que estén en estado SUSP. READY y luego en caso de que no se pueda iniciar ninguno de esos, se pasará a ver los que se encuentren en el estado NEW de acuerdo al algoritmo definido, si los hubiere.

Luego de la finalización, se debe loguear las métricas de estado con el formato adecuado.

Planificador de Corto Plazo

Los procesos que estén en estado READY serán planificados mediante uno de los siguientes algoritmos:

- FIFO
- SJF sin Desalojo
- SJF con Desalojo (Shortest Remaining Time)

FIFO

Se elegirá al siguiente proceso a ejecutar según su orden de llegada a READY.

SJF sin Desalojo

Se elegirá el proceso que tenga la ráfaga más corta. Su funcionamiento será como se explica en teoría y la función de cómo calcular las ráfagas es la siguiente

$Est(n)$ = Estimado de la ráfaga anterior

$R(n)$ = Lo que realmente ejecutó de la ráfaga anterior en la CPU

$Est(n + 1)$ = El estimado de la próxima ráfaga

$$Est(n + 1) = \alpha R(n) + (1 - \alpha) Est(n) \quad \alpha \in [0, 1]$$

SJF con Desalojo

Funciona igual que el anterior con la variante que al ingresar un proceso en la cola de Ready existiendo al menos un proceso en Exec, se debe evaluar si dicho proceso tiene una ráfaga más corta que los que se encuentran en ejecución. En caso de ser así, se debe informar al CPU que posee al Proceso con el tiempo más alto que debe desalojar al mismo para que pueda ser planificado el nuevo.

Ejecución

Una vez seleccionado el proceso a ejecutar, se lo transicionará al estado EXEC y se enviará a uno de los módulos CPU (conectados y libres) el PID asociado a ejecutar a través del puerto de dispatch,

quedando a la espera de recibir dicho PID después de la ejecución junto con un motivo por el cual fue devuelto.

En caso que el algoritmo requiera desalojar al proceso en ejecución, se enviará una interrupción a través de la conexión de interrupt para forzar el desalojo del mismo.

Al recibir el PID del proceso en ejecución, en caso de que el motivo de devolución implique replanificar, se seleccionará el siguiente proceso a ejecutar según indique el algoritmo. Durante este período la CPU se quedará esperando.

Syscalls

Procesos

Dentro de las syscalls que se pueden atender referidas a procesos, tendremos la instrucción `INIT_PROC`.

INIT_PROC, esta syscall recibirá 2 parámetros de la CPU, el primero será el nombre del archivo de pseudocódigo que deberá ejecutar el proceso y el segundo parámetro es el tamaño del proceso en Memoria. El Kernel creará un nuevo PCB y lo dejará en estado NEW.

Memoria

En este apartado solamente se tendrá la instrucción **DUMP_MEMORY**. Esta syscall le solicita a la memoria, junto al PID que lo solicitó, que haga un Dump del proceso.

Esta syscall bloqueará al proceso que la invocó hasta que el módulo memoria confirme la finalización de la operación, en caso de error, el **proceso** se enviará a EXIT. Caso contrario, se desbloquea normalmente pasando a READY.

Entrada Salida

El módulo Kernel administra los distintos dispositivos de IO que se conecten a él. Al momento de que un módulo de IO se conecte al Kernel, este último deberá identificarlo por medio de su nombre, en este punto el Kernel deberá conocer todos los módulos de IO conectados, qué procesos están ejecutando IO en cada módulo y todos los procesos que están esperando una IO determinada.

Para la utilización de estos módulos existirá la syscall **"IO"** que recibirá dos parámetros: el nombre de la entrada/salida a usar y la cantidad de milisegundos que deberá usarla. Primero se deberá validar que la IO solicitada exista en el sistema, si no existe ninguna IO en el sistema con el nombre solicitado, el proceso se deberá enviar a EXIT. En caso de que si exista al menos una instancia de IO, aun si la misma se encuentre ocupada, el kernel deberá pasar el proceso al estado BLOCKED y agregarlo a la cola de bloqueados por la IO solicitada, en caso de que el kernel tenga una instancia de esta IO libre, deberá enviarle al módulo correspondiente el PID y el tiempo por el cual debe bloquearse. Al momento que se conecte una nueva IO o se reciba el desbloqueo por medio de una de ellas, se deberá verificar si hay proceso encolados para dicha IO y enviarlo a la misma.

Al momento de recibir un mensaje de una IO se deberá verificar que el mismo sea una confirmación de fin de IO, en caso afirmativo, se deberá validar si hay más procesos esperando realizar dicha IO, en caso de que el mensaje corresponda a una desconexión de la IO, el proceso que estaba ejecutando en dicha IO, se deberá pasar al estado EXIT.

Planificador de Mediano Plazo

Al ingresar un proceso al estado BLOCKED se deberá iniciar un timer el cual se encargará de esperar un tiempo determinado por archivo de configuración, al terminar ese tiempo si el proceso continúa en estado BLOCKED, él mismo deberá transicionar al estado SUSP. BLOCKED. En este momento se debe informar al módulo memoria que debe ser movido de memoria principal a swap. Cabe aclarar que en este momento vamos a tener más memoria libre en el sistema por lo que se debe verificar si un nuevo proceso puede entrar (tanto de la cola NEW como de SUSP. READY).

Cuando el módulo IO informe que el proceso en SUSP. BLOCKED finalizó su IO correspondiente, el mismo deberá cambiar al estado SUSP. READY y deberá quedar a la espera de su oportunidad de pasar al estado READY.

Una vez que el proceso llegue a SUSP. READY tendrá el mismo comportamiento que la cola NEW teniendo más prioridad que esta última. De esta manera, ningún proceso que esté esperando en la cola de NEW podrá ingresar al sistema si hay al menos un proceso en SUSP. READY. La lógica para tomar estos procesos es FIFO.

Logs mínimos y obligatorios

Syscall recibida: “## (<PID>) - Solicitó syscall: <NOMBRE_SYSCALL>”

Creación de Proceso: “## (<PID>) Se crea el proceso - Estado: NEW”

Motivo de Bloqueo: “## (<PID>) - Bloqueado por IO: <DISPOSITIVO_IO>”

Fin de IO: “## (<PID>) finalizó IO y pasa a READY”

Desalojo de SJF: “## (<PID>) - Desalojado por fin de SJF”

Fin de Proceso: “## (<PID>) - Finaliza el proceso”

Métricas de Estado: “## (<PID>) - Métricas de estado: NEW (NEW_COUNT) (NEW_TIME), READY (READY_COUNT) (READY_TIME), ...”

Archivo de Configuración

Campo	Tipo	Descripción
IP_MEMORIA	String	IP a la cual se deberá conectar con la memoria
PUERTO_MEMORIA	Numérico	Puerto al cual se deberá conectar con la memoria
ALGORITMO_PLANIFICACION	String	FIFO/SJF/SRT

TIEMPO_SUSPENSION	Numérico	Tiempo en milisegundos que se debe esperar antes de mover un proceso al estado SUSP. BLOCKED.
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level from string()

Ejemplo de Archivo de Configuración

```

IP_MEMORIA=127.0.0.1
ALGORITMO_PLANIFICACION=FIFO
PUERTO_MEMORIA=8002
TIEMPO_SUSPENSION=4500
LOG_LEVEL=TRACE

```

Módulo: IO

El módulo IO, en nuestro contexto de TP, va a ser el encargado de simular los diferentes dispositivos de IO que puedan existir en el Sistema.

Lineamiento e Implementación

El módulo **IO** es el encargado de simular las operaciones de Entrada/Salida, para ello al iniciar deberá leer como parámetro de entrada el nombre el cual identificará a la interfaz de IO.

```
➔ ~ ./bin/io [nombre]
```

Una vez leído el nombre, se conectará al Kernel y en el handshake inicial le enviará su nombre para que él lo pueda identificar y quedará esperando las peticiones del mismo.

Al momento de recibir una petición del Kernel, el módulo deberá iniciar un `usleep` por el tiempo indicado en la request.

Al finalizar deberá informar al Kernel que finalizó la solicitud de I/O y quedará a la espera de la siguiente petición.

Logs mínimos y obligatorios

Inicio de IO: “## PID: <PID> - Inicio de IO - Tiempo: <TIEMPO_IO>”.

Finalización de IO: “## PID: <PID> - Fin de IO”.

Archivo de Configuración

Campo	Tipo	Descripción
IP_KERNEL	String	IP a la cual se conectará con el módulo Kernel
PUERTO_KERNEL	Numérico	Puerto al cual se conectará con el módulo Kernel
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level_from_string()

Ejemplo de Archivo de Configuración

```
IP_KERNEL=127.0.0.1
PUERTO_KERNEL=8002
LOG_LEVEL=TRACE
```

Módulo: CPU

El módulo CPU, en nuestro contexto de TP, lo que va a hacer es simular los pasos del ciclo de instrucción de una CPU real, de una forma mucho más simplificada.

Lineamiento e Implementación

El módulo **CPU** es el encargado de interpretar y ejecutar las instrucciones recibidas por parte de la **Memoria**. Para ello, ejecutará un ciclo de instrucción simplificado que cuenta con los pasos: Fetch, Decode, Execute y Check Interrupt.

Las CPUs deberán conectarse al Kernel

Al momento de recibir un **PID** y un **PC** de parte del Kernel la CPU deberá solicitarle a la Memoria la siguiente instrucción para poder iniciar su ejecución.

A la hora de ejecutar instrucciones que requieran interactuar directamente con la Memoria, tendrá que traducir las *direcciones lógicas* (propias del proceso) a *direcciones físicas* (propias de la memoria). Para ello simulará la existencia de una MMU.

Ciclo de Instrucción

Fetch

La primera etapa del ciclo consiste en buscar la próxima instrucción a ejecutar. En este trabajo práctico cada instrucción deberá ser pedida al módulo Memoria utilizando el *Program Counter* (también llamado *Instruction Pointer*) que representa el número de instrucción a buscar relativo al hilo en ejecución.

Decode

Esta etapa consiste en interpretar qué instrucción es la que se va a ejecutar y si la misma requiere de una traducción de dirección lógica a dirección física.

Ejemplos de instrucciones a interpretar

```
1 NOOP
2 WRITE 0 EJEMPLO_DE_ENUNCIADO
3 READ 0 20
4 GOTO 0
5 IO 25000
6 INIT_PROC proceso1 256
7 DUMP_MEMORY
8 EXIT
```


Las instrucciones detalladas previamente son a modo de ejemplo, su ejecución no necesariamente sigue alguna lógica ni funcionamiento correcto. Al momento de realizar las pruebas, ninguna instrucción contendrá errores sintácticos ni semánticos.

Execute

En este paso se deberá ejecutar lo correspondiente a cada instrucción:

- **NOOP**: Representa la instrucción No Operation, es decir, va a consumir solamente el tiempo del ciclo de instrucción.
- **WRITE** (Dirección, Datos): Escribe los datos del parámetro datos en la dirección física obtenida a partir de la dirección lógica que se encuentra en el parámetro Dirección, el campo datos será siempre una cadena de caracteres **sin espacios**.
- **READ** (Dirección, Tamaño): Lee el valor de memoria correspondiente a la dirección física obtenida a partir de la dirección lógica que se encuentra en el parámetro Dirección, de un tamaño determinado por el parámetro Tamaño y lo imprime por pantalla y en el **Log Obligatorio** correspondiente.
- **GOTO** (Valor): Actualiza el PC o Program Counter del proceso al valor pasado por parámetro.

Las siguientes instrucciones se considerarán *Syscalls*, ya que las mismas no pueden ser resueltas por la CPU y depende de la acción del Kernel para su realización, a diferencia de la vida real donde la llamada es a una única instrucción, para simplificar la comprensión de los scripts, vamos a utilizar un nombre diferente para cada *Syscall*.

- **IO** (Tiempo)
- **INIT_PROC** (Archivo de instrucciones, Tamaño)
- **DUMP_MEMORY**
- **EXIT**

Es importante tener en cuenta que al finalizar el ciclo de instrucción el Program Counter (PC) deberá ser actualizado sumándole 1 en caso de que este no haya sido modificado por la instrucción GOTO.

Check Interrupt

En este momento, se deberá chequear si el **Kernel** nos envió una *interrupción* al PID que se está ejecutando, en caso afirmativo, se devuelve el PID y el Program Counter (PC) actualizado al Kernel con *motivo* de la interrupción. Caso contrario, se descarta la interrupción.

MMU

A la hora de traducir **direcciones lógicas a físicas**, la CPU debe tomar en cuenta que el esquema de la memoria de este sistema es de Paginación Multi-nivel, por lo tanto, las direcciones lógicas se compondrán de la siguiente manera:

[entrada_nivel_1 | entrada_nivel_2 | ... | entrada_nivel_X |
desplazamiento]

Estas traducciones, a diferencia de lo que se hace en los ejercicios prácticos que se ven en clases y se toman en los parciales, no se realizarán en binario, ya que es más cómodo utilizar números enteros en sistema decimal para ver los valores, y tomando en cuenta que todas las tablas de todos los niveles tendrán la misma cantidad de entradas, la operatoria sería más parecida a la siguiente.

Teniendo una cantidad de niveles **N** y un identificador **X** de cada nivel podemos utilizar las siguientes fórmulas:

nro_página = floor(dirección_lógica / tamaño_página)

entrada_nivel_X = floor(nro_página / cant_entradas_tabla ^ (N - X)) % cant_entradas_tabla

desplazamiento = dirección_lógica % tamaño_página

TLB

Como las tablas de páginas están presentes en el módulo Memoria, se implementará una TLB para agilizar la traducción de las direcciones lógicas a direcciones físicas.

La TLB contará con la siguiente estructura base: [página | marco], pudiendo agregar campos extra para facilitar la implementación de los algoritmos.

La cantidad de entradas y el algoritmo de reemplazo de la TLB se indicarán por archivo de configuración de la CPU. La cantidad de entradas de la TLB será un entero (pudiendo ser 0, lo cual la deshabilitará), mientras que los algoritmos podrán ser FIFO o LRU.

Al momento de obtener el número de página se deberá consultar en la TLB si se tiene la información de la misma. En caso afirmativo (TLB Hit) se deberá devolver la dirección física (o frame) correspondiente, en caso contrario, se deberá informar el TLB Miss y se deberá consultar a la memoria para obtener el frame correspondiente a la página buscada. Por último, se agregará la nueva entrada a la TLB, si la misma se encuentra llena se deberá reemplazar siguiendo el algoritmo configurado pudiendo elegir como víctima cualquier otra entrada.

Al momento de desalojar un proceso, todas las páginas que se encuentran en la TLB se deberán eliminar.

Caché de páginas

Dado que esta CPU es una simulación de una CPU real, en lugar de tener un tamaño definido en MB y diferentes niveles de Cache (ya que en los procesadores reales tenemos Cache L1, L2, L3, etc), para nuestro trabajo práctico vamos a tener una versión simplificada, donde la caché de páginas va a estar definida por una cantidad de páginas que se podrán almacenar en esta caché. La estructura básica para esta cache deberá ser [página | contenido], pudiendo agregar campos extra para facilitar la implementación de los algoritmos.

La cantidad de entradas y el algoritmo de reemplazo de la caché de páginas se indicarán por archivo de configuración de la CPU. La cantidad de entradas de la caché de páginas será un entero (pudiendo ser 0, lo cual la deshabilitará), mientras que los algoritmos podrán ser CLOCK o CLOCK-M.

Al momento de acceder a una página, la CPU deberá validar si la Caché de páginas se encuentra habilitada (tiene al menos 1 frame) y en caso de que así sea, no debe acceder directamente a las páginas en memoria, si no que deberá hacer todas las operaciones en su Caché, mientras que si la misma está deshabilitada, ahí sí deberá realizar todas las operaciones de lectura y escritura directamente en la memoria principal.

Al momento de cargar una página en la Caché, puede darse el caso de que la caché se encuentre llena y por lo tanto se deberá seleccionar una víctima a ser reemplazada, al momento de reemplazar una página, si la misma fue modificada mientras estuvo cargada en caché, los cambios deberán enviarse a la memoria principal para que esta última tenga la versión actualizada de los datos.

Al momento de desalojar un proceso, todas las páginas que se encuentran en caché en caso de que hayan sido modificadas, deberán actualizarse en la memoria principal y luego se eliminarán todas las entradas de la caché.

De esta manera, al momento de acceder a una página primero se deberá verificar la caché de páginas, luego la TLB y como última instancia la tabla de páginas en memoria.

Logs mínimos y obligatorios

Fetch Instrucción: “## PID: <PID> - FETCH - Program Counter: <PROGRAM_COUNTER>”.

Interrupción Recibida: “## Llega interrupción al puerto Interrupt”.

Instrucción Ejecutada: “## PID: <PID> - Ejecutando: <INSTRUCCION> - <PARAMETROS>”.

Lectura/Escritura Memoria: “PID: <PID> - Acción: <LEER / ESCRIBIR> - Dirección Física: <DIRECCION_FISICA> - Valor: <VALOR LEIDO / ESCRITO>”.

Obtener Marco: “PID: <PID> - OBTENER MARCO - Página: <NUMERO_PAGINA> - Marco: <NUMERO_MARCO>”.

TLB Hit: “PID: <PID> - TLB HIT - Pagina: <NUMERO_PAGINA>”

TLB Miss: “PID: <PID> - TLB MISS - Pagina: <NUMERO_PAGINA>”

Página encontrada en Caché: “PID: <PID> - Cache Hit - Pagina: <NUMERO_PAGINA>”

Página faltante en Caché: “PID: <PID> - Cache Miss - Pagina: <NUMERO_PAGINA>”

Página ingresada en Caché: “PID: <PID> - Cache Add - Pagina: <NUMERO_PAGINA>”

Página Actualizada de Caché a Memoria: “PID: <PID> - Memory Update - Página: <NUMERO_PAGINA> - Frame: <FRAME_EN_MEMORIA_PRINCIPAL>”

Archivo de Configuración

Campo	Tipo	Descripción
IP_MEMORIA	String	IP a la cual se deberá conectar con la memoria

PUERTO_MEMORIA	Numérico	Puerto al cual se deberá conectar con la memoria
IP_KERNEL	String	IP a la cual se deberá conectar con el Kernel
PUERTO_KERNEL_DISPATCH	Numérico	Puerto en el cual se conectará al Kernel para mensajes de dispatch
PUERTO_KERNEL_INTERRUPT	Numérico	Puerto en el cual se conectará al Kernel para mensajes de interrupciones
ENTRADAS_TLB	Numérico	Cantidad de entradas de la TLB
REEMPLAZO_TLB	String	Algoritmo de reemplazo para las entradas de la TLB. FIFO o LRU.
ENTRADAS_CACHE	Numérico	Cantidad de entradas de la Caché de páginas
REEMPLAZO_CACHE	String	Algoritmo de reemplazo para las entradas de la Caché de páginas. CLOCK o CLOCK-M.
RETARDO_CACHE	Numérico	Tiempo en milisegundos que se deberá esperar antes de responder a las solicitudes.
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level from string()

Ejemplo de Archivo de Configuración

```

IP_MEMORIA=127.0.0.1
PUERTO_MEMORIA=8002
PUERTO_ESCUCHA_DISPATCH=8001
PUERTO_ESCUCHA_INTERRUPT=8005
ENTRADAS_TLB=4
REEMPLAZO_TLB=LRU
ENTRADAS_CACHE=2
REEMPLAZO_CACHE=CLOCK
RETARDO_CACHE=250
LOG_LEVEL=TRACE

```

Módulo: Memoria + SWAP

Este módulo será el encargado de responder los pedidos realizados por la CPU para leer y/o escribir en una tabla de páginas, en una porción del espacio de usuario de la memoria o para administrar el pseudocódigo de los procesos que se ejecutarán en el sistema.

A su vez, será el encargado de manejar un espacio de SWAP que se encontrará abstraído en un archivo para cada proceso, de forma tal que se le permite al mismo manejar un mayor nivel de direccionamiento.

Lineamiento e Implementación

Al iniciar levantará un servidor *multihilo* para esperar las peticiones por parte del módulo Kernel y de las múltiples CPU.

Memoria de Sistema

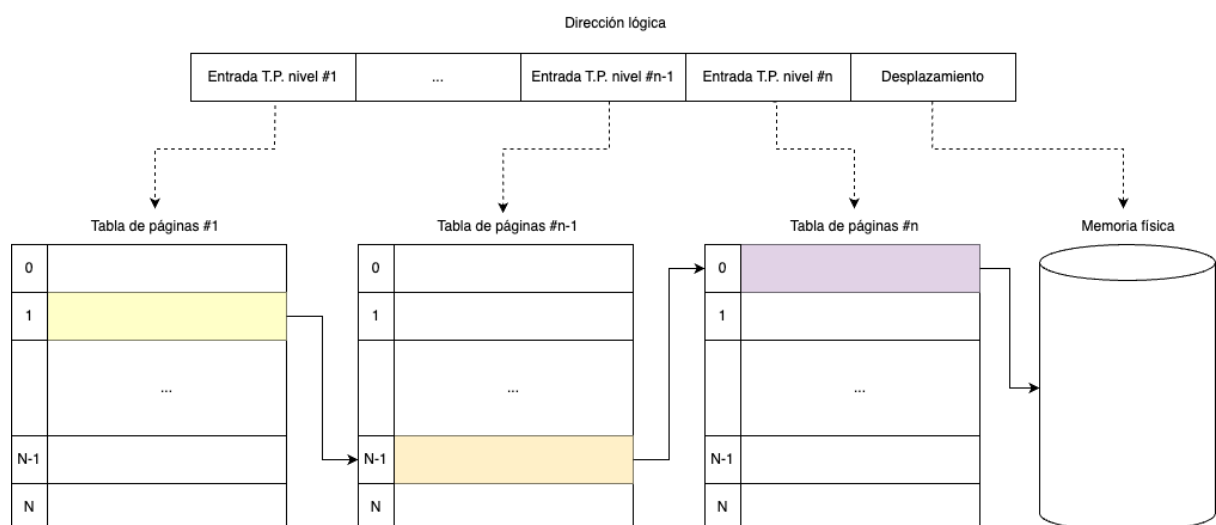
Archivos de pseudocódigo

Por cada PID del sistema, se deberá leer su archivo de pseudocódigo y guardar de forma estructurada las instrucciones del mismo para poder devolverlas una a una a pedido de la CPU. Queda a criterio del grupo utilizar la estructura que crea conveniente para este caso de uso.

Memoria de Usuario

Esquema de memoria

La asignación de memoria de este trabajo práctico utilizará un esquema de **paginación jerárquica multinivel**, donde las tablas de cada nivel contarán con la misma cantidad de entradas, definida por archivo de configuración.



Consideraciones teóricas del esquema

Como se aclara al comienzo de este documento, los conceptos reflejados son, en general, versiones simplificadas o alteradas de los componentes reales de hardware y de sistemas operativos vistos en las clases. Particularmente para este esquema planteado, en sistemas reales normalmente se ve combinado con Memoria Virtual, donde se manifiestan sus principales ventajas. Por otro lado, la cantidad de niveles de paginación normalmente no es libremente configurable sino que depende de la especificación de cada Procesador e implementación del Sistema Operativo.

Este trabajo simplifica el esquema al no combinarlo con Memoria Virtual para enfocarse solamente en sus estructuras y principales diferencias o desventajas con respecto a paginación de un nivel.

Invitamos al alumnado a investigar sobre este esquema en sistemas reales, tanto en el ámbito académico como en procesadores y sistemas operativos modernos, así como reflexionar y discutir acerca de sus beneficios para el aprovechamiento de la memoria.

Estructuras

La memoria contará principalmente con 3 estructuras:

- Un espacio contiguo de memoria (representado por un void*). Este representará el espacio de usuario de la misma, donde los procesos podrán leer y/o escribir.
- Las tablas de páginas, que representarán el espacio de Kernel.
- Bitmap del SWAP.
- Un archivos SWAP denominado `swapfile.bin`.
- Listado de métricas por proceso

Métricas por proceso

El módulo memoria llevará un listado de métricas de cada proceso a fin de poder dar seguimiento a la cantidad de operaciones realizadas por cada proceso en la memoria, por lo tanto el Listado de métricas por proceso tendrá los siguientes valores:

- Cantidad de accesos a Tablas de Páginas
- Cantidad de Instrucciones solicitadas
- Cantidad de bajadas a SWAP
- Cantidad de subidas a Memoria Principal o al espacio contiguo de memoria
- Cantidad de Lecturas de memoria
- Cantidad de Escrituras de memoria

Estas métricas serán informadas al finalizar/destruir un proceso de la memoria principal. La forma de informarlas será por medio de los logs obligatorios con el formato indicado en dicha sección.

Comunicación con Kernel y CPU

Inicialización del proceso

En caso de que se cuente con el tamaño necesario, el módulo memoria deberá crear las estructuras administrativas necesarias y enviar como *respuesta* el número de tabla de página de primer nivel de ese proceso.

Suspensión de proceso

Al ser suspendido un proceso, se debe liberar su espacio en memoria, escribiendo en SWAP solamente la información necesaria. Se debe tener en cuenta que, para la realización de este trabajo práctico, no se requiere el swapeo de tablas de páginas.

Finalización de proceso

Al ser finalizado un proceso, se debe liberar su espacio de memoria y marcar como libres sus entradas en SWAP.

El tiempo de retardo de SWAP definido por archivo de configuración, al tener como objetivo simular la diferencia de velocidades al reemplazar páginas modificadas debido a tener que realizar un acceso extra a disco por escritura, **no debe ser tenido en cuenta para las operaciones con Kernel previamente definidas.**

A su vez, se deberá generar el log obligatorio con las métricas del proceso antes definidas.

Acceso a tabla de páginas

El módulo deberá responder lo siguiente:

- Si se trata de una tabla intermedia (que no es el último nivel), el número de la tabla del siguiente nivel.
- Si se trata de una tabla de último nivel, el número de marco correspondiente.

Acceso a espacio de usuario

El módulo deberá realizar lo siguiente:

- Ante un pedido de lectura, devolver el valor que se encuentra en la posición pedida.
- Ante un pedido de escritura, escribir lo indicado en la posición pedida. En caso satisfactorio se responderá un mensaje de 'OK'.

Leer Página completa

Se deberá devolver el contenido correspondiente de la página a partir del byte enviado como dirección física dentro de la Memoria de Usuario, que deberá coincidir con la posición del byte 0 de la página.

Actualizar página completa

Se escribirá la página completa a partir del byte 0 que igual será enviado como *dirección física*, esta operación se realizará dentro de la Memoria de Usuario y se responderá como OK.

Memory Dump

Al momento de recibir la operación de memory dump el módulo memoria deberá crear un nuevo archivo con el tamaño total de la memoria reservada por el proceso y debe escribir en dicho archivo todo el contenido actual de la memoria del mismo.

El archivo debe llamarse “<PID>-<TIMESTAMP>.dmp” dentro del path definido por archivo de configuración.

Manejo de SWAP

Como se mencionó anteriormente el submódulo de SWAP va a ser el encargado dentro de la memoria de manejar las peticiones de lectura y escritura en SWAP.

Vamos a contar con un archivo denominado *swapfile.bin*, cuya ubicación se va a encontrar definida por archivo de configuración. Este archivo se utilizará para la suspensión de los procesos y contendrá la información de las páginas del proceso que fue suspendido, es responsabilidad del grupo definir la o las estructuras necesarias para poder saber donde se encuentran las páginas de cada proceso en SWAP ya que las mismas en algún momento deberán volver a cargarse en memoria principal para reanudar la ejecución del proceso.

Es importante aclarar que al momento de suspender un proceso no es necesario que se guarden en SWAP sus tablas de páginas.

Logs mínimos y obligatorios

Conexión de Kernel¹: “## Kernel Conectado - FD del socket: <FD_DEL_SOCKET>”

Creación de Proceso: “## PID: <PID> - Proceso Creado - Tamaño: <TAMAÑO>”

Destrucción de Proceso: “## PID: <PID> - Proceso Destruído - Métricas - Acc.T.Pag: <ATP>; Inst.Sol.: <Inst.Sol.>; SWAP: <SWAP>; Mem.Prin.: <Mem.Prin.>; Lec.Mem.: <Lec.Mem.>; Esc.Mem.: <Esc.Mem.>”

Obtener instrucción: “## PID: <PID> - Obtener instrucción: <PC> - Instrucción: <INSTRUCCIÓN> <...ARGS>”

Escritura / lectura en espacio de usuario²: “## PID: <PID> - <Escritura/Lectura> - Dir. Física: <DIRECCIÓN_FÍSICA> - Tamaño: <TAMAÑO>”

Memory Dump: “## PID: <PID> - Memory Dump solicitado”

¹ Los FD al ser un valor int, se pueden imprimir %d, para más información sobre los FD, ver la [Guía de Sockets](#)

² En el caso de leer o escribir una página entera el tamaño es el tamaño de página y la dirección es el byte 0 de la página.

Archivo de Configuración

Campo	Tipo	Descripción
PUERTO_MEMORIA	Numérico	Puerto al cual se deberá conectar con la memoria
TAM_MEMORIA	Numérico	Tamaño expresado en bytes del espacio de usuario de la memoria.
TAM_PAGINA	Numérico	Tamaño de las páginas en bytes.
ENTRADAS_POR_TABLA	Numérico	Cantidad de entradas de cada tabla de páginas.
CANTIDAD_NIVELES	Numérico	Cantidad de niveles de tablas de páginas
RETARDO_MEMORIA	Numérico	Tiempo en milisegundos que debiera esperarse antes de responder una petición de memoria
PATH_SWAPFILE	String	Path donde se encuentra el archivo de swapfile.bin
RETARDO_SWAP	Numérico	Tiempo en milisegundos que deberá esperarse antes de responder una petición de SWAP
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level from string()
DUMP_PATH	String	Path donde se almacenarán los archivos de DUMP

Ejemplo de Archivo de Configuración

```
PUERTO_MEMORIA=8002
TAM_MEMORIA=4096
TAM_PAGINA=64
ENTRADAS_POR_TABLA=4
ENTRADAS_POR_TABLA=5
RETARDO_MEMORIA=1500
PATH_SWAP=/home/utnso/swapfile.bin
RETARDO_SWAP=15000
LOG_LEVEL=TRACE
DUMP_PATH=/home/utnso/dump_files/
```

Descripción de las entregas

Debido al orden en que se enseñan los temas de la materia en clase, los checkpoints están diseñados para que se pueda realizar el trabajo práctico de manera iterativa incremental tomando en cuenta los conceptos aprendidos hasta el momento de cada checkpoint.

Check de Control Obligatorio 1: Conexión inicial

Fecha: 19/04/2025

Objetivos:

- Familiarizarse con Linux y su consola, el entorno de desarrollo y el repositorio.
- Aprender a utilizar las Commons, principalmente las funciones para listas, archivos de configuración y logs.
- Definir el Protocolo de Comunicación.
- Todos los módulos están creados y son capaces de establecer conexiones entre sí.

Check de Control Obligatorio 2: Ejecución Básica

Fecha: 24/05/2025

Objetivos:

- **Módulo Kernel:**
 - Planificación de corto y largo plazo con FIFO
 - Administración de IO
- **Módulo CPU:**
 - Interpretación de instrucciones y ejecución de instrucciones
- **Módulo Memoria:**
 - Devolver lista de instrucciones
 - Devolver un valor fijo de espacio libre (mock)
- **Módulo IO: Completo**

Carga de trabajo estimada:

- **Módulo Kernel:** 40%
- **Módulo CPU:** 30%
- **Módulo Memoria:** 15%
- **Módulo IO:** 15%

Check de Control Obligatorio 3: CPU Completa y Memoria

Fecha: 21/06/2025

Objetivos:

- **Módulo Kernel:**
 - Planificación de corto y largo plazo completa.
- **Módulo CPU: Completo**
- **Módulo Memoria:**
 - Esquema de Memoria principal completo
 - Administración de espacio libre en memoria principal

Carga de trabajo estimada:

- **Módulo Kernel:** 30%
- **Módulo CPU:** 30%
- **Módulo Memoria:** 40%

Entregas Finales

Fechas: 12/07/2025, 19/07/2025, 02/08/2025

Objetivos:

- Finalizar el desarrollo de todos los procesos.
- Probar de manera intensiva el TP en un entorno distribuido.
- Todos los componentes del TP ejecutan los requerimientos de forma integral.